

Note 4.2 Integer Representations and Algorithms

Introduction

In this section we will introduce the representations of integer and show related algorithms.

Representation of Integers

Theorem 1

Let b be an integer greater than 1, then if n is a positive integer, it can be expressed uniquely in the form

$$n = a_k b^k + a_{k-1} b^{k-1} + \cdots + a_1 b + a_0$$

where k is a nonnegative integer, $a_0, a_1, \dots, a_k$ are nonnegative integers less than b and $a_k \neq 0$.

The proof can be done by using mathematical induction. The representation of n given in Theorem 1 is called the base b expansion of n , denoted by $(a_k a_{k-1} \cdots a_1 a_0)_b$. And typically the subscript 10 can be omitted for decimal expansions.

Binary Expansions

Choose 2 as the base gives the binary expansions. And it is used by computers to represent and do arithmetic with integers.

Octal and Hexadecimal Expansions

Base 8 expansions are called octal expansions and base 16 expansions are hexadecimal expansions, use the number and the letters from a to f to represent 10 to 15.

Bytes are bit strings of length eight and can be represented by two hexadecimal digits. ($2^8 = 16^2$)

Base Conversion

To construct base b expansion of an integer n , we first divide n by b to obtain the quotient and the remainders, that is

$$n = bq_0 + a_0, 0 \leq a_0 < b$$

then the remainder a_0 is the rightmost digit in the base b expansion of n . Then divide q_0 by b to obtain the quotient and the remainders. Continue this process until the quotient is zero.

ALGORITHM 1 Constructing Base b Expansions.

```
procedure base  $b$  expansion( $n, b$ : positive integers with  $b > 1$ )  
 $q := n$   
 $k := 0$   
while  $q \neq 0$   
     $a_k := q \bmod b$   
     $q := q \div b$   
     $k := k + 1$   
return  $(a_{k-1}, \dots, a_1, a_0)$   $\{(a_{k-1} \dots a_1 a_0)_b$  is the base  $b$  expansion of  $n\}$ 
```

The algorithm can be thought of as a greedy algorithm.

Conversion Between Binary, Octal and Hexadecimal Expansions

To converse between binary and octal and between binary and hexadecimal expansions, we need to know that each octal digit corresponds to a block of three binary digits and each hexadecimal digit corresponds to a block of four binary digits.

Algorithms for Integer Operations

Suppose that the binary expansion of a and b are

$$a = (a_{n-1}a_{n-2} \dots a_1a_0)_2, b = (b_{n-1}b_{n-2} \dots b_1b_0)_2$$

Addition Algorithm

For addition algorithm, we need to begin at the rightmost bit, then for every bit, we have that

$$a_k + b_k + c_{k-1} = c_k \cdot 2 + s_k$$

where $k \geq 1$ and s_k is the bit in the binary expansion of $a + b$. For $k = 0$, $a_0 + b_0 = c_0 \cdot 2 + s_0$.

Then the procedure produces the binary expansion of the sum namely

$$a + b = (s_n s_{n-1} s_{n-2} \dots s_1 s_0)_2$$

ALGORITHM 2 Addition of Integers.

```
procedure add( $a, b$ : positive integers)  
{the binary expansions of  $a$  and  $b$  are  $(a_{n-1}a_{n-2} \dots a_1a_0)_2$   
 and  $(b_{n-1}b_{n-2} \dots b_1b_0)_2$ , respectively}  
 $c := 0$   
for  $j := 0$  to  $n - 1$   
     $d := \lfloor (a_j + b_j + c)/2 \rfloor$   
     $s_j := a_j + b_j + c - 2d$   
     $c := d$   
 $s_n := c$   
return  $(s_0, s_1, \dots, s_n)$  {the binary expansion of the sum is  $(s_n s_{n-1} \dots s_0)_2$ }
```

And the time complexity of this algorithm is $O(n)$.

Multiplication Algorithm

Using the distributive law, we see that

$$\begin{aligned} ab &= a(b_0 2^0 + b_1 2^1 + \cdots + b_{n-1} 2^{n-1}) \\ &= a(b_0 2^0) + a(b_1 2^1) + \cdots + a(b_{n-1} 2^{n-1}) \end{aligned}$$

Then if $b_j = 0$, then $a(b_j 2^j) = 0$. If not, $a(b_j 2^j) = (ab_j) 2^j = a \cdot 2^j$. Then we can obtain the result by **shifting** the binary expansion of a j places to the left, adding j zero bits at the tail end. Then obtain ab by adding the n integers $ab_j 2^j, j = 0, 1, \dots, n-1$.

ALGORITHM 3 Multiplication of Integers.

```

procedure multiply( $a, b$ : positive integers)
{the binary expansions of  $a$  and  $b$  are  $(a_{n-1}a_{n-2} \dots a_1a_0)_2$ 
and  $(b_{n-1}b_{n-2} \dots b_1b_0)_2$ , respectively}
for  $j := 0$  to  $n-1$ 
    if  $b_j = 1$  then  $c_j := a$  shifted  $j$  places
    else  $c_j := 0$ 
{ $c_0, c_1, \dots, c_{n-1}$  are the partial products}
 $p := 0$ 
for  $j := 0$  to  $n-1$ 
     $p := p + c_j$ 
return  $p$  { $p$  is the value of  $ab$ }

```

Since for $ab_j 2^j, j = 0, 1, \dots, n-1$ requires at most

$$0 + 1 + 2 + \cdots + n - 1$$

shifts. Hence the number of shifts required is $O(n^2)$.

Algorithm For div and mod

ALGORITHM 4 Computing div and mod.

```

procedure division algorithm( $a$ : integer,  $d$ : positive integer)
 $q := 0$ 
 $r := |a|$ 
while  $r \geq d$ 
     $r := r - d$ 
     $q := q + 1$ 
if  $a < 0$  and  $r > 0$  then
     $r := d - r$ 
     $q := -(q + 1)$ 
return  $(q, r)$  { $q = a \text{ div } d$  is the quotient,  $r = a \text{ mod } d$  is the remainder}

```

And we need $O(n^2)$ bit operations here.

For $a < 0$, actually $-a = q \cdot d + r$, thus $a = -q \cdot d - r = -(q + 1) \cdot d + (d - r)$. Thus $r := d - r, q := -(q + 1)$

Modular Exponentiation

To find $b^n \bmod m$, we need to transfer the power into binary expansion, that is,

$$b^n = b^{a_{k-1}2^{k-1} + \dots + a_1 \cdot 2 + a_0} = b^{a_{k-1}2^{k-1}} \dots b^{a_1 \cdot 2} \cdot b^{a_0}$$

Then the algorithm successively finds $b \bmod m, b^2 \bmod m, \dots, b^{2^{k-1}} \bmod m$ and multiplies together those terms where $a_j = 1$. Then finding the remainder of the product after each multiplication. It uses the [corollary](#).

ALGORITHM 5 Modular Exponentiation.

```
procedure modular exponentiation( $b$ : integer,  $n = (a_{k-1}a_{k-2} \dots a_1a_0)_2$ ,  
     $m$ : positive integers)  
 $x := 1$   
 $power := b \bmod m$   
for  $i := 0$  to  $k - 1$   
    if  $a_i = 1$  then  $x := (x \cdot power) \bmod m$   
     $power := (power \cdot power) \bmod m$   
return  $x$  { $x$  equals  $b^n \bmod m$ }
```

And it uses $O(\log^2 m \log n)$ bit operations.